

## **EXPERIMENT – 8**

**Name:** K ANUSRI

**Roll No:** 2520030432

**Section:** 8

### **Title**

### **Demonstration of Copy-on-Write Behavior after fork()**

### **Aim**

To develop a program that demonstrates Copy-on-Write behavior after a fork() and analyze memory usage before and after modifying data in the child process.

### **Theory**

**Copy-on-Write (COW)** is a memory-management technique used after fork(). The parent and child initially share the same memory pages. When the child modifies a shared page, the operating system creates a separate copy of that page for the child.

Thus, modifications made by the child do not affect the parent's data.

### **Algorithm**

1. Start the program.
2. Allocate memory for a data array.
3. Initialize the data in the allocated memory.
4. Display the memory usage before fork().
5. Create a child process using fork().
6. In the child process, display the memory usage before modification.
7. Modify the allocated data in the child process.
8. Display the memory usage after modification.
9. Display the modified child data.
10. In the parent process, wait for the child process to complete.
11. Display the parent's data to verify that it was not modified by the child.
12. Display the memory usage of the parent after the child modification.
13. Free the allocated memory.
14. Stop the program.

### **Program Code**

```
k_anusri@DESKTOP-TM20BAK × + ▾
k_anusri@DESKTOP-TM20BAK:~$
k_anusri@DESKTOP-TM20BAK:~$ cat > exp8_cow.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int *data = malloc(5 * sizeof(int));

    if (data == NULL)
    {
        perror("malloc");
        return 1;
    }

    for (int i = 0; i < 5; i++)
        data[i] = 10;

    printf("Before fork - Parent data: ");
    for (int i = 0; i < 5; i++)
        printf("%d ", data[i]);
    printf("\n");

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        free(data);
        return 1;
    }

    if (pid == 0)
    {
        printf("Child before modification: ");
        for (int i = 0; i < 5; i++)
            printf("%d ", data[i]);
        printf("\n");

        data[0] = 99;

        printf("Child after modification: ");
        for (int i = 0; i < 5; i++)
            printf("%d ", data[i]);
        printf("\n");

        free(data);
    }
    else
    {
        wait(NULL);

        printf("Parent after child modification: ");
        for (int i = 0; i < 5; i++)
            printf("%d ", data[i]);
        printf("\n");

        free(data);
    }

    return 0;
}
```

## Execution Screenshots

#### Screenshot 1 – Compilation and Execution

```
k_anusri@DESKTOP-TM20BAQ:~$ gcc exp8_cow.c -o exp8_cow
k_anusri@DESKTOP-TM20BAQ:~$ ./exp8_cow
Child before modification: 10 10 10 10 10
Child after modification: 99 10 10 10 10
Parent after child modification: 10 10 10 10 10
```

#### Screenshot 2 – Copy-on-Write Demonstration

```
k_anusri@DESKTOP-TM20BAQ:~$ gcc exp8_cow.c -o exp8_cow
k_anusri@DESKTOP-TM20BAQ:~$ ./exp8_cow
Before fork - Parent data: 10 10 10 10 10
Child before modification: 10 10 10 10 10
Child after modification: 99 10 10 10 10
Parent after child modification: 10 10 10 10 10
```

#### Result

The program successfully demonstrated **Copy-on-Write behavior after fork()**. The child process modified its copy of the data while the original data in the parent process remained unchanged. Memory usage before and after the child's modification was observed.